

CSCE 735 PARALLEL COMPUTING · MODULE 13 · LECTURE NOTES

Correctness, Debugging, and Shipping

The parts that determine whether any of it is usable.

MODULE OUTCOMES

- Explain why testing establishes less for concurrent code.
- Use the tools that detect possible rather than observed failures.
- Achieve reproducibility and know what it costs.
- Debug a parallel program systematically.
- State what you can honestly claim about parallel code.

1 What testing establishes

PASSING TESTS SHOWS THAT ONE INTERLEAVING WORKED

A serial program follows one execution path for a given input. Running it with that input establishes what happens for that input, which is a great deal — and it is why testing works as well as it does for sequential code.

A concurrent program has exponentially many possible interleavings of its threads' operations, and a single run exercises exactly one of them — whichever the scheduler, the cache state, and the ambient system load happened to produce.

So a thousand passing runs establish that a thousand interleavings worked, drawn from a space you cannot enumerate and which is typically vastly larger.

And the untested interleavings are not randomly distributed. They are the ones that arise under heavy load, on different hardware, with a different thread count — which is to say, in production. **Testing concentrates its sampling in precisely the conditions where the bugs are not.**

Technique	What it establishes	Limitation
Testing	This particular interleaving produced the right answer.	Very weak, per the callout above.
Dynamic race detection (ThreadSanitizer)	No data race was <i>possible</i> on the paths executed — it tracks the happens-before relation rather than observing failures, so it reports races that did not actually occur.	Only covers code paths actually executed, so coverage still matters. The practical sweet spot , and the reason it should run over the whole test suite.

Technique	What it establishes	Limitation
Model checking (SPIN, TLA+)	No interleaving of the model violates the specified properties. Exhaustive.	Small models only — the state space explodes. And the result applies to the model, not the code.
Static analysis with annotations	A specific class of error is absent, at compile time.	False positives, and the annotation burden is real.
Formal proof	The algorithm is correct, given the model.	Expensive , and the model may not match the hardware — several proofs assumed sequential consistency (Module 04).

MODEL CHECK THE SMALL CORE

Tools such as SPIN and TLA+ explore every possible interleaving of a model exhaustively, and report a counterexample trace when a property fails — which is a qualitatively different kind of result from testing.

They cannot handle a whole program. The state space grows combinatorially, and any realistic application exceeds it by many orders of magnitude.

But a lock protocol, a concurrent queue, a cache coherence handshake, or a state machine is small enough — and those are exactly the components where the hard bugs live. The surrounding application code is usually straightforwardly sequential.

So model the tricky core and test the rest. Several published lock-free algorithms had bugs discovered this way (Module 05), years after publication and peer review, which is the strongest available evidence that the technique finds things human review does not.

2 Reproducibility

PARALLEL RESULTS ARE NOT BITWISE REPRODUCIBLE BY DEFAULT

Floating-point addition is not associative (Module 06 §3), so the result of a reduction depends on the order in which the partial sums are combined.

And that order depends on the thread count, the schedule, and the timing. With dynamic scheduling or atomic accumulation, two runs on the *same* machine with the *same* input can produce different results in the last bits.

This is not a bug and it must be expected. It is a consequence of how floating-point arithmetic works combined with how parallel reduction works, and no implementation quality removes it.

If you need bitwise reproducibility — for regression testing, for debugging (where a changing answer makes bisection impossible), or for a scientific result someone else must replicate — **you must fix the combination order and pay for it.** CSCE 649's Module 13 reached the same conclusion from the simulation side, and for the same underlying reason.

- **A fixed decomposition independent of the thread count.** The same chunks, combined in the same order, regardless of how many threads execute them.
- **Deterministic reduction trees** — combine partial results in a predetermined pattern rather than in whatever order threads finish.
- **No atomics for accumulation**, since the order in which atomic updates land is non-deterministic by construction.
- **Compensated summation** (Kahan, or Neumaier) tracks and corrects the rounding error. **This reduces the error rather than fixing the order**, which is a different goal and often the more useful one — a more accurate answer may matter more than an identical one.
- **And it costs performance.** A fixed combination order forbids dynamic load balancing (Module 06), so a workload with variable per-item cost pays in imbalance. **Reproducibility and load balancing are in direct tension**, and which you need is a decision rather than a default.

3 Debugging

- 1 **Reproduce it.** Record the thread count, the input, and any relevant environment. **Then make it more likely:** increase the thread count beyond the core count, inject random delays at suspicious points, and run under load. All of these widen the windows in which the bad interleaving occurs.
- 2 **Does it fail with one thread?** If it does, it is not a concurrency bug at all, and the entire problem becomes an ordinary sequential debugging exercise. **This is the step people skip, and it resolves a surprising fraction of supposed concurrency bugs** — an uninitialised variable or an off-by-one does not become a race because the program has threads.
- 3 **Run the sanitisers.** ThreadSanitizer, AddressSanitizer, and compute-sanitizer for GPU code. They frequently identify the problem immediately, with both stack traces.
- 4 **Reduce it.** Cut the program down — remove threads, remove phases, shrink the input — until the bug disappears. **The last thing you removed is the clue**, and a minimal reproducer is worth more than any amount of reasoning about the full program.
- 5 **Then reason.** With a small enough example, the specific interleaving can be held in your head and the argument can be made properly. **Reasoning first, on the full program, is what wastes days.**

HEISENBUGS: OBSERVING CHANGES THE SCHEDULE

Adding a print statement changes the timing, which changes the interleaving, which frequently hides the bug entirely — and then removing the print brings it back, which is a demoralising experience.

Running under a debugger does the same, more dramatically. Single-stepping one thread serialises the execution completely, and breakpoints change the relative timing by orders of magnitude.

So use low-overhead logging into a per-thread ring buffer, dumped after the failure. No locks, no I/O during the run, and minimal disturbance to timing — a few nanoseconds per entry rather than microseconds.

Record a timestamp and a thread identifier with every entry, so the interleaving can be reconstructed afterwards from the merged logs. **Reconstructing the schedule after the fact is the technique**; observing it as it happens changes what you are observing.

4 Claiming honestly

- **'It is X times faster than the optimised serial baseline, on this hardware, at this problem size, with these compiler flags.'** With the distribution rather than the minimum (Module 07 §3). Every one of those qualifiers is load bearing.
- **'It scales to N cores, departing from ideal at this point, for this reason.'** Naming the reason — bandwidth saturation, load imbalance, synchronisation — is what distinguishes a measurement from a graph.
- **'It is bandwidth-bound at an arithmetic intensity of I.'** With the roofline placement to support it (Module 07).

- **'ThreadSanitizer reports no data races over our test suite.'** Not **'the code is race-free'** — the tool examined the paths your tests exercised, and that is exactly what the claim should say. **This phrasing is the model for all of them:** state what was established, not what you hope follows from it.
- **And what you cannot honestly say: 'it is correct'.** You can say what it passes, what was checked, and by what method. For concurrent code that gap is wider than for sequential code, and pretending otherwise is how the bugs reach production.

WHERE THIS LEAVES YOU

You can take a working program and determine, by measurement, what limits it — and therefore which optimisations can possibly help and which cannot. That diagnostic ability is worth more than any individual technique in this course, because it is what prevents weeks spent on the wrong thing.

You can write correct shared-memory, GPU, and distributed-memory code, and you know which tools establish what about its correctness — and how much less that is than you would like.

And you know the three things that reorient most people's expectations: most real code is bandwidth-bound rather than compute-bound; most published speedup claims are overstated, usually through the baseline; and most parallel bugs are design failures rather than coding errors.

CSCE 629 chose the algorithm. CSCE 614 explained the machine. CSCE 647, 649, 608, 650, and 748 supplied the workloads that needed the speed. This course made them fast — and, more importantly, honestly measured, which is the part that survives when the hardware changes.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Lamport — TLA+ and the Specifying Systems book (free)	Model checking concurrent algorithms. The book is free and the video course is a reasonable entry point.
Holzmann — The SPIN Model Checker	The tool and its documentation, free. Well suited to the small-core modelling of §1.
ThreadSanitizer and compute-sanitizer documentation (free)	What each tool establishes and what it does not — worth reading the limitations sections specifically.
Hoefler & Belli — Scientific Benchmarking of Parallel Computing Systems (free)	The reporting discipline of §4, made rigorous. The single most useful paper in this course for anyone who will publish performance results.
Demmel & Nguyen — reproducible floating-point summation (free)	The §2 problem and the algorithms that address it.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Take a concurrent data structure and run it a million times. Then run it under ThreadSanitizer and compare what each found.
- 2 **Model a lock protocol or a bounded queue in TLA+ or SPIN** and check a safety property. Then introduce a bug and confirm it finds it.
- 3 **Demonstrate non-reproducibility:** run a parallel float reduction twenty times at several thread counts and report every distinct answer.
- 4 Implement a deterministic reduction and confirm bitwise reproducibility across thread counts. Measure the performance cost.
- 5 Implement Kahan summation and compare the error against both the naive parallel and the serial result.
- 6 Take a concurrency bug and apply the five-step method. Document what each step contributed.
- 7 **Create a Heisenbug:** a race that disappears when a print statement is added. Then find it with a per-thread ring buffer.
- 8 Write a per-thread logging buffer with timestamps and reconstruct an interleaving from it.
- 9 Take a performance claim from a paper or a README and rewrite it to meet §4's standard. Note what information is missing.
- 10 **Project 2 is now due.** Submit the implementation, the measured optimisation steps including any that failed, the profiler evidence, the library comparison, and the statement of what limits the final version.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why does testing establish less for concurrent code than for sequential code?
- 2 Compare five verification techniques on what each establishes and its limit.
- 3 What makes dynamic race detection stronger than testing?
- 4 What can model checking do, what can it not, and what follows?
- 5 Why are parallel floating-point results not reproducible?
- 6 Give four requirements for reproducibility and the cost.
- 7 Give the five debugging steps and say which is most often skipped.
- 8 What is a Heisenbug, and what technique replaces printf debugging?
- 9 Give four honest performance claims and one thing you cannot say.